

DATA STRUCTURES AND PROBLEM SOLVING

Colored Water Sort Solver Project

C++17 breadth-first search solver with a native Windows dashboard

Course	Data Structures and Problem Solving
Student 1	[NAME AND ID REQUIRED]
Student 2	[NAME AND ID OR N/A REQUIRED]
Repository	github.com/yousefgharib335/Colored-Water-Sort-Solver-Project
Report status	Draft - student details and presentation link pending
Date	15 August 2026

1. Executive Summary

This project implements the Colored Water Sort puzzle as a C++17 console application. It models each arrangement as a search state and uses breadth-first search (BFS) to explore legal one-layer transfers. Because BFS visits states in increasing path length, the first solution found has the minimum number of moves. A native Win32 dashboard is included as an optional bonus interface for entering puzzles, loading randomized solvable examples, and examining each solution step visually.

Requirements coverage

Requirement	Implementation evidence	Status
C++ console solver	src/main.cpp; command-line input and output	Complete
Minimum-move solution	BFS with visited-state tracking and parent links	Complete
No-solution reporting	Frontier exhaustion returns No solution exists.	Complete
Sample inputs	solvable, no_solution, and already_solved	Complete
GUI bonus	src/gui_main.cpp native Windows dashboard	Complete
Public repository	Repository exists; local upload still pending	Pending
Recorded presentation	Script prepared; recording URL needed	Pending

Submission note

The LMS submission should contain only this final PDF. Before submission, replace all red pending fields with the confirmed team information and recorded presentation URL.

2. Team Responsibilities

The specification permits one or two students and requires both students to make meaningful commits and speak in the presentation when a two-person team is used. The following allocation is a practical draft and must be confirmed before submission.

Member	Proposed responsibilities	Evidence to retain
Student 1 - [NAME/ID]	Console solver, BFS state search, input validation, console tests	Multiple focused commits and test outputs
Student 2 - [NAME/ID or N/A]	Windows GUI, randomized examples, report, presentation/demo preparation	Multiple focused commits and GUI demonstration
Shared	Review rules, verify minimum moves, record final demo, approve report	Final checklist and recording

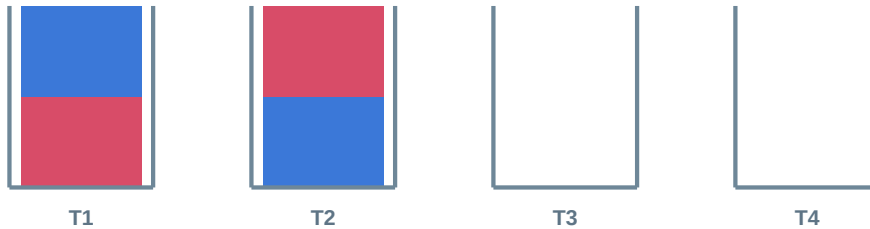
If this is a one-student project, change the table to one member and combine all responsibilities. If it is a two-student project, both accounts must contribute meaningful commits rather than one final upload.

Development scope

- Required console interface: completed in `src/main.cpp`.
- Bonus graphical interface: completed in `src/gui_main.cpp` using the native Windows API.
- Testing assets: three specification-focused input files plus compiled local executables.
- Documentation: README build/run guide, this report, and a timed presentation script.

3. Problem Definition and Design

A puzzle contains N tubes of equal capacity C . Values are entered bottom-to-top. Positive integers are colors and 0 marks unused capacity. A legal move removes exactly one top layer from a nonempty source and places it into a destination that is not full and is either empty or has the same top color.



Solved-state interpretation

Every nonempty tube must be uniform, and each color must be consolidated into only one tube. Empty tubes are allowed. A uniform tube may be partially full if that color has fewer than C layers in the input.

Input validation

- N and C must be positive integers.
- Every console tube line contains exactly C nonnegative integers.
- Zeros may appear only after all colored layers in a tube.
- The GUI accepts omitted trailing rows or values as empty spaces and enforces a display limit of 18 tubes and capacity 12.
- The BFS search can become expensive before the GUI's display limits because the number of reachable states grows combinatorially.

Move rule used by the program

One transferred layer equals one move. A contiguous block is not transferred as a single move; each layer is counted separately. This matches the program output and minimum-move search.

4. Data Structures

Data structure	Role	Why selected
vector (Tube)	Stores colors bottom-to-top; back() is the top layer	Dynamic size, indexed access, $O(1)$ amortized push/pop at the top
vector (State)	Stores the complete arrangement	Simple deep copy and deterministic traversal of tubes
queue	BFS frontier of node indices	FIFO order guarantees increasing move depth
unordered_map	Maps encoded states to visited node indices	Average $O(1)$ duplicate detection and prevents cycles
vector	Stores state, parent index, incoming move, and parent flag	Stable numeric parent references and efficient path reconstruction
unordered_set	Tracks colors already consolidated in solved-state validation	Average $O(1)$ duplicate-color check

State encoding

Each state is serialized with tube separators and color separators, for example `|1,2,|2,1,||`. The separators preserve tube boundaries and make the key unambiguous for the visited map.

Parent-link reconstruction

Rather than copying an entire move sequence into every frontier entry, each search node stores only the index of its parent and the move used to reach it. Once BFS finds a solved state, the program follows parent indices back to the initial state and reverses the collected moves.

5. Breadth-First Search Algorithm

The initial state is inserted into the node list, FIFO frontier, and visited map. The solver repeatedly removes the oldest frontier node, checks whether it is solved, and generates every ordered source/destination pair. Legal and previously unseen results are appended to the frontier.

```
BFS(initial):
enqueue initial; mark initial visited
while frontier is not empty:
current = dequeue frontier
if current is solved:
reconstruct and return its move path
for every source tube S:
for every destination tube D where S != D:
next = copy(current)
if one legal layer can move S -> D
and next has not been visited:
store next with parent=current and move=(S,D)
mark visited; enqueue next
return no solution
```

Why the result is minimum

All edges represent one layer transfer and therefore have equal cost. BFS explores all states at distance d before any state at distance $d+1$. Consequently, when a solved state is first removed from the frontier, no solution with fewer moves can exist.

Why the solver terminates

The puzzle has a finite set of arrangements. Each arrangement is inserted into the visited map at most once. BFS therefore either finds a solution or exhausts every reachable state and reports that no solution exists.

GUI random examples

The dashboard shuffles complete color sets into initially filled tubes, keeps two empty tubes, rejects already-solved arrangements, and runs the same BFS solver before displaying an example. Thus every generated example shown to the user is unsolved and verified solvable.

6. Time and Memory Complexity

Let N be the number of tubes, C the capacity, and V the number of distinct reachable states explored before completion.

Operation	Cost	Reason
Solved-state check	$O(NC)$	At most C layers are inspected in each of N tubes
Ordered move pairs per state	$O(N^2)$	Every source is paired with every different destination
Copy and encode one candidate	$O(NC)$	The full vector-of-vectors state is copied and serialized
Worst-case search time	$O(V N^3 C)$	$O(N^2)$ candidates, each with an $O(NC)$ state copy/encoding
Search memory	$O(VNC)$	Nodes and visited keys retain up to V complete states
Solution reconstruction	$O(M)$	M parent links are followed for an M -move answer

Practical behavior

V can grow exponentially or combinatorially with puzzle size, so the practical limit is determined by the arrangement, available memory, and search depth rather than only by N and C . The console parser does not impose a fixed maximum, while the GUI intentionally caps display input at 18 tubes and capacity 12. Large unsolved or deeply scrambled puzzles may still require excessive time or memory far below those values.

Implementation note

The complexity shown here reflects the current implementation, which copies a complete state for every ordered source/destination candidate. A future optimization could test move legality before copying, use a compact numeric encoding, and apply safe symmetry reduction.

7. Code and File Structure

Path	Purpose
src/main.cpp	Required console parser, legal-move logic, BFS solver, path reconstruction, and output
src/gui_main.cpp	Bonus Win32 dashboard, shared solver logic, random examples, guide, drawing, and navigation
samples/solvable.txt	Official solvable four-tube example
samples/no_solution.txt	Official valid puzzle with no solution
samples/already_solved.txt	Zero-move solved-state edge case
CMakeLists.txt	C++17 console build configuration with warnings
gui/CMakeLists.txt	Windows GUI build and user32/gdi32 linkage
README.md	Puzzle interpretation, build/run commands, algorithm, and GUI summary

Why this structure was chosen

The required console application remains independent and easy to grade. The optional GUI is isolated in a separate translation unit and build configuration so it does not add platform dependencies to the console target. Sample inputs are plain text and can be redirected directly into the executable for repeatable tests.

Build targets

`water_sort_solver` is portable C++17. `water_sort_gui` is a Windows subsystem executable that links against `user32` and `gdi32`.

8. Test Results

The console executable was run on 15 August 2026 using the three sample files in the repository. All observed results matched the expected behavior.

Case	Input summary	Observed result	Assessment
1. Solvable	N=4, C=2; [1,2], [2,1], empty, empty	Solution found; minimum 3 moves; final [1,1], empty, [2,2], empty	Pass
2. No solution	N=2, C=2; [1,2], [2,1]	No solution exists.	Pass
3. Already solved	N=3, C=2; [1,1], [2,2], empty	Solution found; minimum 0 moves; state unchanged	Pass

Verified move sequence for Case 1

1. Tube 1 -> Tube 3
2. Tube 2 -> Tube 1
3. Tube 2 -> Tube 3

Testing interpretation

- Case 1 checks BFS minimum-path behavior and legal transfers into empty and matching-color tubes.
- Case 2 checks complete frontier exhaustion when no destination space exists.
- Case 3 checks the goal test at depth zero and confirms that an already-solved puzzle needs no transfers.
- The final state prints zeros for unused capacity so the output remains in the required fixed-width tube format.

9. Bonus Windows Dashboard

The GUI is an optional enhancement and does not replace the required console application. It uses only the C++ standard library and native Win32/GDI functions, so no third-party GUI framework is required.

Feature	Behavior
Solve puzzle	Parses the editor input and displays the minimum move list or no-solution result
Random example	Generates a different unsolved puzzle and validates solvability with BFS
Solution explorer	Draws tubes and lets the user move backward, forward, or directly to the final state
Guide and rules	Explains input order, move rules, solved state, and practical/display limits
Responsive layout	Centers dashboard panels and adjusts drawing dimensions with the window size
Input assistance	Missing GUI rows/values are empty; invalid negative or misplaced-zero inputs are rejected

Limits

The GUI parser accepts at most 18 tubes and capacity 12 for display usability. These are interface limits, not a promise that BFS will solve every puzzle at that size. State-space growth is the governing practical constraint.

Color visualization

The dashboard maps integer color codes onto a rotating palette and labels each visible layer when space permits. Codes beyond the palette length reuse colors visually, while the solver still treats their integer values as distinct.

10. Build, Run, and Submission Links

Console build and run

```
cmake -S . -B build
cmake --build build

Get-Content samples/solvable.txt | .\build\water_sort_solver.exe
```

Windows GUI build and run

```
cmake -S gui -B gui/build
cmake --build gui/build

.\gui\build\water_sort_gui.exe
```

GitHub repository

<https://github.com/yousefgharib335/Colored-Water-Sort-Solver-Project>

Recorded presentation

[RECORDED PRESENTATION URL REQUIRED BEFORE SUBMISSION]

Final submission checklist

- Confirm student names, IDs, and responsibility allocation on the cover and Section 2.
- Ensure both students have meaningful GitHub commits if this is a two-person team.
- Make the GitHub repository public and verify it opens while signed out.
- Record a presentation of no more than 10 minutes; both students must speak when applicable.
- Insert the presentation URL into this report and verify link permissions.
- Upload only the final PDF to LMS from one team member's account.

Conclusion

The core project requirements are implemented and validated. BFS guarantees a minimum-length solution under the one-layer-per-move rule, duplicate states are avoided, and parent links reconstruct the exact move sequence. The bonus dashboard provides an accessible visual demonstration without changing the required console interface.

End of report